实验一 python 实验环境与信息熵的计算实现

学号：11260211041 姓名：蔺天震 电话：18562520379

一、环境搭建

1、实验基础环境信息

使用个人笔记本进行实验，MacBook Pro，芯片 Apple M1 Pro，内存 16G，操作系统 macOS Tahoe 26.0.1

开发环境使用 VScode 1.141.0 (Universal)，Python 版本 3.10

2、下载 miniconda

MacBook 中打开终端，执行以下命令：

```
lintianzhen@lintianzhendeMacBook-Pro ~ % curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest
-MacOSX-arm64.sh

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  162M  100  162M    0     0   102k      0  0:27:04  0:27:04 --:--:--  842k
```

图 1 下载 Miniconda

3、执行安装

```
lintianzhen@lintianzhendeMacBook-Pro ~ % bash Miniconda3-latest-MacOSX-arm64.sh

Welcome to Miniconda3 py314_26.9.3-0

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
```

图 2 安装 Miniconda

4、验证安装

```
(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda --version
conda 26.9.3
(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda info

     active environment : base
    active env location : /Users/lintianzhen/miniconda3
            shell level : 1
       user config file : /Users/lintianzhen/.condarc
 populated config files : /Users/lintianzhen/miniconda3/.condarc
                          /Users/lintianzhen/miniconda3/condarc.d/anaconda-auth.
yml
                          /Users/lintianzhen/.condarc
          conda version : 26.9.3
    conda-build version : not installed
         python version : 3.14.8.final.0
                 solver : libmamba (default)
       virtual packages : __archspec=1=m1
```

图 3 验证安装

5、设置为国内清华源

```
(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda config --show channels
channels:
  - defaults
(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda config --add channels http
s://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r

conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/m
sys2
conda config --set show_channel_urls yes

(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda config --show channels
channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - defaults
```

图 4 设置 conda 国内源

6、创建 conda 环境

创建了一个名为 ml2026 的 conda 环境，Python 基础版本为 3.10

```
(base) lintianzhen@lintianzhendeMacBook-Pro ~ % conda create -n ml2026 python=3.10
Channels:
 - defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /Users/lintianzhen/miniconda3/envs/ml2026

  added / updated specs:
    - python=3.10


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    bzip2-1.0.8                |       h80987f9_6         129 KB  defaults
    ca-certificates-2026.9.25  |       hca03da5_0         107 KB  defaults
    libcxx-22.1.2              |       hf894667_0         309 KB  defaults
```

图 5 创建 conda 环境

7、激活 conda 环境

```
(base) lintianzhen@lintianzhendeMacBook-Pro 机器学习I % conda activate ml2026
(ml2026) lintianzhen@lintianzhendeMacBook-Pro 机器学习I %
```

图 6 激活 conda 环境

二、信息熵计算小程序

1、信息熵的定义以及计算方法

$$H(X) = -\sum_{i=1}^{n} p_i \log_2 p_i$$，p_i 为第 i 个信息

出现的概率。

2、程序实现思路

1）输入一段文字，文字可以包含中文、英文、字符等；

2）对输入的文字进行分词操作，得到分词列表和每个词在文字中出现的概率；

3）分别对每个词进行计算，汇总得到该段文字的信息熵。

4）添加测试函数，对 5 段文字进行分别求信息熵，输出分词结果，每个词出现的概率和最终的信息熵。

3、基础框架选择

1）分词选择 jieba 分词,使用 pip install 命令安装依赖；

```
(ml2026) lintianzhen@lintianzhendeMacBook-Pro 机器学习I % pip3 install jieba -i https://mirrors.ustc.edu.cn/pypi/simple
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple
Collecting jieba
  Downloading jieba-0.42.1.tar.gz (19.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.2/19.2 MB 241.1 kB/s  0:01:27
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: jieba
  Building wheel for jieba (pyproject.toml) ... done
  Created wheel for jieba: filename=jieba-0.42.1-py3-none-any.whl size=19314509 sha256=f15f441775cdd43b7f1d9787a29ae0c01c0c936a5276d1dd461189d81036bef0
  Stored in directory: /Users/lintianzhen/Library/Caches/pip/wheels/ef/1b/80/e8bbbfe2ff9d39057572ea2882ffd2d84493550e7c8f00521b
Successfully built jieba
Installing collected packages: jieba
Successfully installed jieba-0.42.1
```

图 7 安装 jieba 分词

2）求对数计算等数学操作采用 Python 自带的 math 包进行

4、实现代码

```python
import math
import jieba
from collections import Counter

TraeCode Plugin: 解释代码 | 注释代码 | ✕ | </> | 解释 | 添加注释 | ✕ | @
def tokenize(text):
    """
    使用 jieba 对输入的文字进行分词操作。
    """
    # 使用 jieba 精确模式分词，返回列表
    words = list(jieba.cut(text))
    return words
```

图 8 引入 jieba 包，定义分词函数

```python
def calculate_text_entropy(text):
    """
    计算一段文字的信息熵。
    1. 分词
    2. 计算每个词的概率
    3. 汇总计算信息熵
    返回：(分词列表, 词概率字典, 信息熵值)
    """
    #  分词
    words = tokenize(text)

    # 剔除中英文标点符号
    punctuation_pattern = re.compile(r'[!"#$%&\'()*+,\-./:;<=>?@[\\\]^_`{|}~，。、；：？！‘’“”（）《》【】…—～·]')
    words = [word for word in words if not punctuation_pattern.search(word)]

    total_words = len(words)
    if total_words == 0:
        return words, {}, 0.0

    # 统计词频并计算每个词的概率
    word_counts = Counter(words)
    word_probs = {}
    for word, count in word_counts.items():
        word_probs[word] = count / total_words

    # 计算信息熵
    entropy = 0.0
    for prob in word_probs.values():
        if prob > 0:
            entropy -= prob * math.log2(prob)

    return words, word_probs, entropy
```

图 9 计算信息熵函数

```python
def test_entropy():
    """
    测试函数：对5段不同的文字分别求信息熵，
    并输出分词结果、每个词出现的概率和最终的信息熵。
    """
    # 5段测试文字
    test_texts = [
        "Hello world! This is a test.",
        "你好，世界！这是一个测试。",
        "信息熵是信息论中的一个重要概念，用来度量信息的不确定性。信息熵越大，不确定性越高。",
        "Python is a great programming language. Python is easy to learn and Python is powerful.2026.10.20",
        "2026年10月11日，央视《新闻直播间》栏目报道了信息科学与工程学部跨学科复合型人才培养实践，展示了学部在交叉学科育人、科研攻关与成果转化等方面的探索与成效。学
    ]

    for i, text in enumerate(test_texts, 1):
        print(f"\n{'='*20} 测试文字 {i} {'='*20}")
        print(f"[原文]: {text}")

        words, probs, entropy = calculate_text_entropy(text)

        print(f"[分词结果]: {words}, 共 {len(words)} 个词")
        print("[每个词的概率]:")
        for word, prob in probs.items():
            print(f"  - '{word}': {prob:.4f} ({prob*100:.2f}%)")
        print(f"[最终信息熵]: {entropy:.4f} bits")
        print("="*55)
```

图 10 信息熵测试函数

5、实验结果

使用 Python 命令运行以上程序，在控制台输出 5 段文本的信息熵值。具体执行输出详见目录内 log 文件。

表 1 实验结果

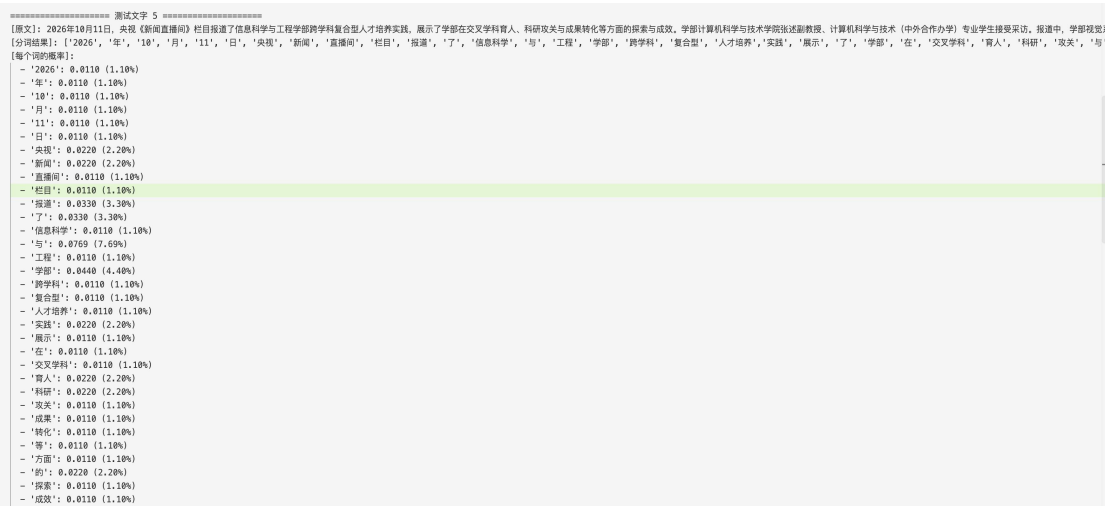
测试数据	分词数	信息熵
测试文本 1	11	2.4040bits
测试文本 2	5	2.3219bits
测试文本 3	17	3.7345bits
测试文本 4	28	2.5640bits
测试文本 5	91	5.9236bits

```
====================== 测试文字 1 ======================
[原文]: Hello world! This is a test.
Building prefix dict from the default dictionary ...
Loading model from cache /var/folders/w4/b88tdcls515201dj1_nhdy7c0000gn/T/jieba.cache
Loading model cost 0.548 seconds.
Prefix dict has been built successfully.
[分词结果]: ['Hello', ' ', 'world', ' ', 'This', ' ', 'is', ' ', 'a', ' ', 'test']，共 11 个词
[每个词的概率]:
  - 'Hello': 0.0909 (9.09%)
  - ' ': 0.4545 (45.45%)
  - 'world': 0.0909 (9.09%)
  - 'This': 0.0909 (9.09%)
  - 'is': 0.0909 (9.09%)
  - 'a': 0.0909 (9.09%)
  - 'test': 0.0909 (9.09%)
[最终信息熵]: 2.4040 bits
============================================================

====================== 测试文字 2 ======================
[原文]: 你好，世界！这是一个测试。
[分词结果]: ['你好', '世界', '这是', '一个', '测试']，共 5 个词
[每个词的概率]:
  - '你好': 0.2000 (20.00%)
  - '世界': 0.2000 (20.00%)
  - '这是': 0.2000 (20.00%)
  - '一个': 0.2000 (20.00%)
  - '测试': 0.2000 (20.00%)
[最终信息熵]: 2.3219 bits
============================================================
```

```
================== 测试文字 3 ==================
[原文]: 信息熵是信息论中的一个重要概念，用来度量信息的不确定性。信息熵越大，不确定性越高。
[分词结果]: ['信息熵', '是', '信息论', '中', '的', '一个', '重要', '概念', '用来', '度量', '信息', '的', '不确定性', '信息熵', '越大', '不确定性', '越高']，共 17 个词
[每个词的概率]:
  - '信息熵': 0.1176 (11.76%)
  - '是': 0.0588 (5.88%)
  - '信息论': 0.0588 (5.88%)
  - '中': 0.0588 (5.88%)
  - '的': 0.1176 (11.76%)
  - '一个': 0.0588 (5.88%)
  - '重要': 0.0588 (5.88%)
  - '概念': 0.0588 (5.88%)
  - '用来': 0.0588 (5.88%)
  - '度量': 0.0588 (5.88%)
  - '信息': 0.0588 (5.88%)
  - '不确定性': 0.1176 (11.76%)
  - '越大': 0.0588 (5.88%)
  - '越高': 0.0588 (5.88%)
[最终信息熵]: 3.7345 bits
============================================================

================== 测试文字 4 ==================
[原文]: Python is a great programming language. Python is easy to learn and Python is powerful.2026.10.20
[分词结果]: ['Python', ' ', 'is', ' ', 'a', ' ', 'great', ' ', 'programming', ' ', 'language', ' ', 'Python', ' ', 'is', ' ', 'easy', ' ', 'to', ' ', 'learn', ' ', 'and', ' ', 'Python', ' ', 'is', ' ']，共 28 个词
[每个词的概率]:
  - 'Python': 0.1071 (10.71%)
  - ' ': 0.5000 (50.00%)
  - 'is': 0.1071 (10.71%)
  - 'a': 0.0357 (3.57%)
  - 'great': 0.0357 (3.57%)
  - 'programming': 0.0357 (3.57%)
  - 'language': 0.0357 (3.57%)
  - 'easy': 0.0357 (3.57%)
  - 'to': 0.0357 (3.57%)
  - 'learn': 0.0357 (3.57%)
  - 'and': 0.0357 (3.57%)
[最终信息熵]: 2.5640 bits
```




```
  - '计算机科学': 0.0220 (2.20%)
  - '技术': 0.0220 (2.20%)
  - '学院': 0.0110 (1.10%)
  - '张述': 0.0110 (1.10%)
  - '副教授': 0.0110 (1.10%)
  - '中外合作': 0.0110 (1.10%)
  - '办学': 0.0110 (1.10%)
  - '专业': 0.0110 (1.10%)
  - '学生': 0.0110 (1.10%)
  - '接受': 0.0110 (1.10%)
  - '采访': 0.0110 (1.10%)
  - '中': 0.0110 (1.10%)
  - '视觉': 0.0110 (1.10%)
  - '系统': 0.0110 (1.10%)
  - '机器人': 0.0110 (1.10%)
  - '实验室': 0.0110 (1.10%)
  - '亮相': 0.0110 (1.10%)
  - '还': 0.0110 (1.10%)
  - '介绍': 0.0110 (1.10%)
  - '董军宇': 0.0110 (1.10%)
  - '教授': 0.0110 (1.10%)
  - '团队': 0.0110 (1.10%)
  - '攻克': 0.0110 (1.10%)
  - '水下': 0.0110 (1.10%)
  - '三维重建': 0.0110 (1.10%)
  - '技术难题': 0.0110 (1.10%)
  - '推动': 0.0110 (1.10%)
  - '相关': 0.0110 (1.10%)
  - '装备': 0.0110 (1.10%)
  - '实现': 0.0110 (1.10%)
  - '国产化': 0.0110 (1.10%)
  - '替代': 0.0110 (1.10%)
  - '将': 0.0110 (1.10%)
  - '产业': 0.0110 (1.10%)
  - '串成': 0.0110 (1.10%)
  - '一条线': 0.0110 (1.10%)
  - '创新': 0.0110 (1.10%)
[最终信息熵]: 5.9236 bits
```

6、总结分析

1) 遇到的问题：使用 jieba 分词后，标点符号也被当做了“词”划分到了分词列表中，使标点符号也参与了概率计算，使得最终得到信息熵偏高。

解决办法：在得到分词列表后，增加了一步过滤操作，使用正则表达式剔除纯标点符号。

2）注意事项：信源熵可以按“字”计算，也可以按“词”计算。考虑实际应用场景，我选择了按“词”来计算，但是信息熵计算准确性依赖分词的准确性，如果分词不准，概率分布会变，熵值也会变。

3）测试用例要多样化，考虑实际应用场景，应包含中文、英文、字符、短文本、长文本等多种场景。也应当考虑边界值和异常处理。